

RESEARCH

Open Access



Detecting advanced persistent threats via heterogeneous graph learning from homophily and heterogeneity views

Yuanhuang Liu^{1,2}, Ayong Ye^{1,2*} , Wenting Lu^{1,2} and Longjing Yang^{1,2}

Abstract

Advanced Persistent Threats (APTs) is one of the most serious cybersecurity threats today, posing a substantial threat to enterprises and organizations due to their stealthy and targeted nature. Data provenance-based methods are widely used for APT detection but often rely on specific rules and high-quality data due to limitations in capturing complete graph structures, reducing their effectiveness in diverse detection environments. To overcome this issue, we propose APT-HERA, a model employs heterogeneous graph representation learning to learn system behavior patterns that can adapt to environments with limited data. The embedding representations of the provenance graph in APT-HERA are derived from both homophily and heterogeneity perspectives, thereby enabling a more comprehensive extraction of the rich structural information contained within the provenance graph. The performance of APT-HERA was evaluated on four public datasets. Experimental results demonstrate that APT-HERA achieves 98% precision in information-constrained detection scenarios, outperforming state-of-the-art methods including MAGIC, Flash, and ThreaTrace under such conditions.

Keywords Advanced persistent threats, Heterogeneous graph, Intrusion detection

Introduction

With the intensification of cybersecurity threats, large-scale enterprises and government agencies have increasingly become prime targets of cyberattacks. APTs characterized by their stealthy, complex, and prolonged nature, often hide in critical internal networks, leading to severe consequences such as data breaches, infrastructure paralysis, and other serious disruptions. For instance, the “SolarWinds” Center for Internet Security (2021) supply chain attack led to the exposure of sensitive information from multiple U.S. federal agencies. Similarly, the “NotPetya” Petya (2024) attack caused

widespread outages across key sectors in Ukraine, crippling government departments, banks, energy companies, and the power grid, resulting in major disruptions across various industries.

APT detection has become an important research topic widely concerned with academia and industry field. A series of traditional attack detection schemes such as statistical analysis Hassan et al. (2019); Liu (2018), learning-based approaches Li et al. (2022); Ding et al. (2023); Hassan et al. (2020); Milajerdi et al. (2019), and static code analysis Laurenza et al. (2017); Bolton and Anderson-Cook (2017) can be employed to detect APT attacks. However, as network boundaries expand and the prevalence of sophisticated malicious activities increases, it is impractical to rely on traditional methods for detecting APT attacks. Data provenance analysis is an effective approach for fine-grained attack detection and analyzing APTs. It converts raw data, like system audit logs into provenance graphs that visually represent the

*Correspondence:

Ayong Ye
yay@fjnu.edu.cn

¹ The College of Computer Science and Network Security, Fujian Normal University, Fuzhou 350117, Fujian, China

² Fujian Provincial Key Laboratory of Network Security and Cryptology, Fujian Normal University, Fuzhou 350117, Fujian, China

interactions and activities of entities within a system. These graphs help identify anomalies, such as abnormal connections or unusual interaction patterns. Despite significant progress in research on APT detection based on data provenance, there are many challenges in the practical application of these methods:

(1) *How to reduce reliance on predefined rules.* Incorporating expert knowledge or specific rules can significantly enhance the modeling of APT attack patterns. Most methods Hassan et al. (2020); Milajerdi et al. (2019); Hossain et al. (2017); Milajerdi et al. (2019); Hossain et al. (2020); Chen et al. (2022); Fang et al. (2022); Xu et al. (2022); Wang et al. (2020); Xiao et al. (2024) relied on rules derived from known APTs patterns and expert knowledge to learn malicious and benign behavior patterns. However, rule-based methods depend on heuristic rules to cover attack patterns, making them less effective when attack behaviors change or new, unknown attacks emerge. For example, in the “SolarWinds” attack Center for Internet Security (2021), attackers randomized parts of their actions making traditional identification steps such as scanning for known indicators of compromise (IOC) of limited value. Additionally, rule definition is not only time-consuming and labor-intensive but also lacks universality, hindering its effective deployment across diverse environments;

(2) *How to reduce reliance on high-quality data.* Abundant raw data is essential for training APT detection models. Many methods Manzoor et al. (2016); Han et al. (2020); Wang et al. (2022); Yang et al. (2023); Zengy et al. (2022); Aly et al. (2024); Jia et al. (2024) depend heavily on large amounts of high-quality historical data, limiting their effectiveness in real-world detection scenarios where such data may not be available. For instance, ProGrapher Yang et al. (2023) and Flash Rehman et al. (2024) enhance detection by incorporating comprehensive semantic information from system log entries. However, in complex and diverse network environments, many scenarios or devices may lack rich semantic data. For example, network traffic data and device logs often only include basic information such as IP addresses, port numbers, and protocol encodings. In such cases, intrusion detection methods that rely heavily on rich high-quality data may suffer from reduced accuracy and increased false alarm rates.

(3) *How to capture complete provenance graph structure information.* Existing methods often depend on external knowledge and high-quality data to learn system behavior patterns due to their inability to capture the full structural features of provenance graphs. Therefore, effectively learning and leveraging the structural features of provenance graphs is crucial to cope with these challenges. Traditional detection methods often overlook the

structural characteristics of graphs. For example, meta-path-based methods Hossain et al. (2020); Chen et al. (2022); Fang et al. (2022); Wang et al. (2020) use predefined attack paths to capture features of intrusion behaviors, excluding benign nodes and edges outside these paths. Similarly, random walk-based methods Xu et al. (2022); Rehman et al. (2024); Liu et al. (2019) for graph embedding only focus on elements within the walk rules, ignoring structural features beyond them. This limitation in capturing the complete graph structure hampers the ability to construct comprehensive behavioral patterns. To deal with this problem, they have to introduce more external information or richer data.

To address these challenges, in this paper, we propose APT-HERA, a novel intrusion detection model that utilizes a self-supervised heterogeneous graph representation learning approach combined with a lightweight classifier to efficiently detect APT attacks. In contrast to previous detection methods Chen et al. (2022); Yang et al. (2023); Aly et al. (2024); Rehman et al. (2024), APT-HERA learns node embeddings from both homophily(connectivity and information aggregation between similar nodes) and heterogeneity(connectivity and information aggregation between different nodes) perspectives of the provenance graph. During the graph representation phase, self-expression matrices are used to capture homophily representations by aggregating information from neighboring nodes of the same type, while heteromorphic representations are obtained by aggregating different types of edge information using a heteromorphic encoder. These representations are then combined to form the final node embeddings. This approach integrates both homophily and heteromorphic structural characteristics, allowing for the extraction of more comprehensive structural information without relying on predefined rules, thereby reducing dependence on high-quality data and enhancing adaptability across detection scenarios. APT-HERA uses a lightweight classifier to categorize node representations as benign or malicious, enabling fine-grained detection of malicious entities. Additionally, a community detection algorithm is employed to recover malicious subgraphs and trace the execution of malicious activities.

In summary, the contributions of our work are as follows:

1. We present APT-HERA, a provenance graph-based APTs detection model that uses heterogeneous graph representation learning and lightweight classifiers for granularity-tunable malicious detection and attack tracing. This model is the first to detect APT attacks from the perspective of both homophily and hetero-

geneity of provenance graphs, achieving high-precision detection with limited data.

2. APT-HERA employs a novel node embedding approach that integrates homophily and heterogeneity of provenance graphs. It aggregates information from similar nodes for homogeneous representations and from different node types for heterogeneity representations, capturing more complex and diverse structural information to learn system behavior patterns, thereby reducing reliance on predefined rules and high-quality data. This approach enhances the model's generalization, enabling it to adapt to different detection environments.
3. We validate the effectiveness and performance of APT-HERA on four widely-used datasets, involving both real-world and stimulated APT attacks. The experimental results show that our method achieves high accuracy and low false alarm rate in various detection scenarios with minimum computation overhead.

Related work

At present, the defense methods against APT attacks can be divided into three categories: rule-based methods, anomaly-based methods, and sequence-based methods. The relevant literature is outlined as follows:

Rule-based approach

Rule-based approaches Hassan et al. (2020); Milajerdi et al. (2019); Hossain et al. (2017); Milajerdi et al. (2019); Hossain et al. (2020); Chen et al. (2022); Fang et al. (2022); Xu et al. (2022); Wang et al. (2020); Xiao et al. (2024) leverage prior knowledge of known attacks to develop specific heuristic rules for detecting malicious behavior. For instance, Holmes Milajerdi et al. (2019) applies expert knowledge of existing TTPs (Tactics, Techniques, and Procedures) to identify potential attacks within traceability graphs, facilitating threat detection, alert generation, and scenario reconstruction. Similarly, Poirot Milajerdi et al. (2019) detects threats by correlating metrics identified by other systems and employs expert insights derived from existing cyber threat reports to construct attack graphs, matching them against provenance graphs for threat detection. The APT-MMF approach Xiao et al. (2024) offers an attribution method for APTs attackers based on multimodal and multilevel feature fusion. This method utilizes heterogeneous attribute graphs to represent APTs cyber threat reports and their associated indicators of compromise.

Certain methods also learn attackers' behavioral patterns by defining predefined attack paths. For example, LMTracker Fang et al. (2022) models malicious behavior

by defining the lateral movement path of an attacker following the compromise of an intranet. Likewise, Prov-Detector Wang et al. (2020) and APT-KGL Chen et al. (2022) employ meta-paths to guide random walk techniques in learning behavioral patterns associated with malicious activities within provenance graphs.

Anomaly-based approach

Anomaly-based approaches Manzoor et al. (2016); Han et al. (2020); Wang et al. (2022); Yang et al. (2023); Zengy et al. (2022); Aly et al. (2024); Jia et al. (2024) involve constructing a model of normal system behavior to identify deviations indicative of anomalies. StreamSpot Manzoor et al. (2016) uses information flow graphs to detect intrusions by extracting graph features to learn a benign model and applying clustering to identify anomalies. Unicorn Han et al. (2020) employs a WL-kernel-based method to extract global graph features and detect anomalies through an evolutionary model, though it is limited in detecting hidden threats due to the constraints of graph kernel methods. ThreaTrace Wang et al. (2022) utilizes GraphSAGE Hamilton et al. (2017) to learn features of benign entities in provenance graphs for malicious entity detection. ProGraphe Yang et al. (2023) combines Graph2Vec Narayanan et al. (2017) and TextRCNN Lai et al. (2015) embeddings to identify graph-level anomalies. ShadeWatcher Zengy et al. (2022) uses Graph Neural Networks (GNNs) to characterize system entities' preferences and performs edge-level anomaly detection to identify potential adversarial interactions. Magic Jia et al. (2024) employs a graph autoencoder to reconstruct node features in provenance graphs, learning benign behavior patterns to detect outliers. Flash Rehman et al. (2024) integrates Word2Vec Church (2017) with GNN to capture semantic information from audit data for malicious detection on provenance graphs and leverages an embedded database to accelerate model detection.

Sequence-based approach

Sequence-based approaches Du et al. (2017); Alsaheel et al. (2021); Zhang et al. (2019); Shen and Stringhini (2019); Ding et al. (2023) extract event sequences from the original graph and classify them as either malicious or benign. For example, ATLAS Alsaheel et al. (2021) employs natural language processing (NLP) and Long Short-Term Memory (LSTM) models to categorize event sequences by distinguishing key characteristics of benign and malicious behaviors. DeepLog Du et al. (2017) also utilizes LSTM to learn typical patterns, identifying anomalies based on deviations from these patterns. Similarly, LogRobust Zhang et al. (2019) and LogGAN Xia et al. (2019) are LSTM-based anomaly detectors; LogRobust uses an attention-based Bi-LSTM model to

capture contextual information in log sequences, effectively identifying unstable log events. Attack2Vec Shen and Stringhini (2019) extracts security events from Intrusion Prevention System (IPS) logs and represents them as phrases, with each sequence as a sentence and each event as a word, employing a word embedding model to understand the evolution of cyber-attacks. AIRTAG Ding et al. (2023) applies NLP techniques and a BERT model to analyze raw logs, capturing features and relationships between event sequences to reconstruct the attack story.

As shown in Table 1, we compared the characteristics of APT-HERA with those of recent related studies. Unlike previous methods, APT-HERA does not rely on specific rules or high-quality data to learn the characteristics of malicious behavior. Instead, it captures rich graph structural information by learning the homophily and heterogeneity of provenance graphs to model interactions between system entities and identify malicious activities. Most existing methods are limited to capturing either heterogeneity or homogeneity, but not both simultaneously. This approach not only improves detection efficiency but also enhances the model's generalization, enabling adaptation to different detection environments.

Background and motivation

Attack scenario

This section we provide an APT attack scenario that we use throughout the paper, illustrated in Fig. 1, based on an example from ThreaTRACE Wang et al. (2022) and derived from the DARPA Engagement 3 Theia Keromytis (2018) dataset. The diagram shows four types of entity nodes, with arrows indicating direct interactions; Red nodes represent entities involved in malicious behavior, while white nodes indicate benign entities. In the scenario, the attacker, identified by IP address $x.x.x.x$, exploits a backdoor vulnerability in the Firefox.exe process to implant a profile file on the victim host and gain root privileges. The process is then executed, establishing a connection to the attacker's system at $y.y.y.y$. Following this, a file with root privileges is implanted and executed. Finally, the attacker uses the mail process to scan port $z.z.z.z$ on the target host for reconnaissance

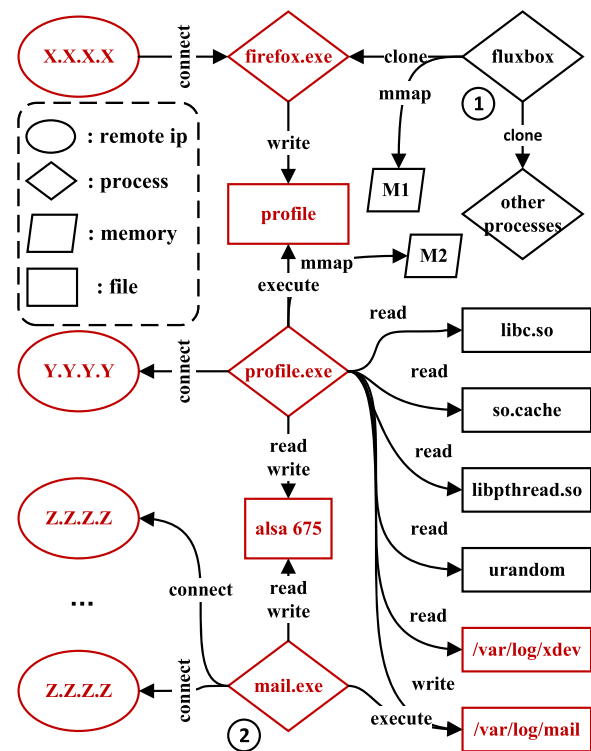


Fig. 1 Example of attack scenario derived from the DARPA Engagement 3 Theia dataset

and to establish a covert link between the attacker and the target.

Motivation

Our research goal is to achieve efficient APT detection and traceability without relying on external information and high-quality data. In this paper, we argue that in an APT attack, despite the attacker's efforts to conceal their intrusion activities, the nodes corresponding to their malicious activities in the provenance graph will still exhibit neighborhood characteristics and interaction behaviors that distinguish them from benign nodes. To illustrate, in Fig. 1, the malicious process node ② engages in numerous interactions with remote IP nodes, and its

Table 1 Comparison of the characteristics of APT-HERA with other methods

Methods	Rule	High-quality data	Homophily	Heterogeneity
ThreaTrace Wang et al. (2022)	✓	✓		✓
Flash Rehman et al. (2024)	✓	✓	✓	
Magic Jia et al. (2024)				✓
APT-KGL Chen et al. (2022)	✓	✓		✓
PROGRAPHER Yang et al. (2023)		✓	✓	
APT-HERA			✓	✓

local structure is markedly disparate from that of the benign process node ①. This attribute motivates us to achieve node-level anomaly detection through the acquisition of structural characteristics inherent to provenance graphs.

The following two insights form the basis of our research:

Insight 1: Homophily. Nodes exhibiting similar behaviors within a local space are closer together in the feature space. Nodes with similar interaction characteristics in the provenance graph are likely to be entities of the same type Grover and Leskovec (2016); Li et al. (2021). To illustrate, in Fig. 1 *mail* and *profile* are both process nodes, and they both have operations that read and write files and interact with remote IP nodes. The linear representation between nodes of the same type is obtained by aggregating information through establishing links between nodes that are characterized by similar attributes, which can capture the homophily of the graph structure. The aggregation of information in a homomorphic manner can effectively extract the intrinsic

consistency of the graph structure and enhance the relevance of nodes of the same type.

Insight 2: Heterogeneity. Different node types in a provenance graph exhibit significant differences in attributes and interaction patterns. As shown in Fig. 1, process nodes and file nodes exhibit distinct interaction behaviors and structural compositions. For instance, the feature vectors for benign node ① and malicious node ② are $[1, 2, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0]$ and $[1, 0, 0, 0, 1247, 1, 1, 1, 1, 0, 0, 0, 1]$, respectively, showing a significant difference in the fourth dimension. Statistical analysis of attribute correlations among major nodes in the provenance graph, using the DARPA Engagement 3 dataset, confirms this observation. Fig. 2 illustrates that feature correlations among major nodes are generally low, indicating significant discrepancies in their attributes. While different node types show varied feature performances, nodes connected within the local space often exhibit some degree of correlation. Aggregating information from different types of edges highlights the graph's heterogeneity, revealing the diversity and complexity of the graph structure and

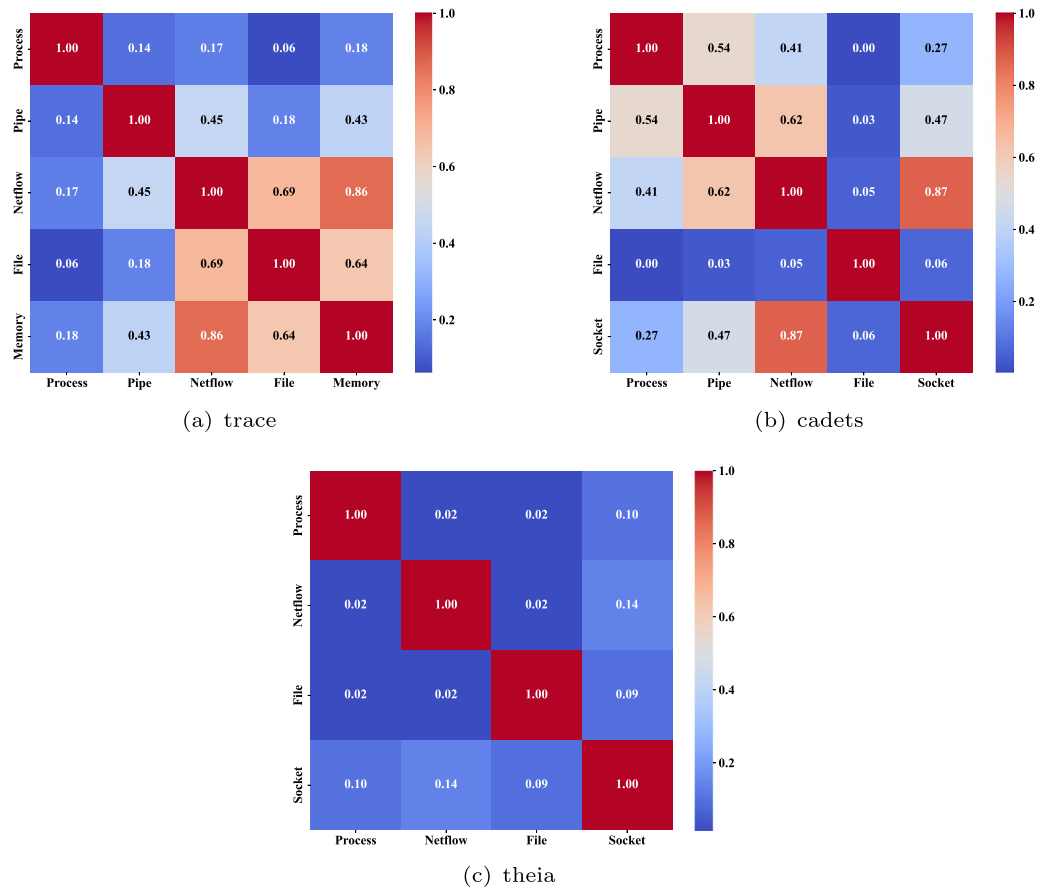


Fig. 2 Feature correlation of different types of nodes in DARPA Engagement 3 dataset

providing insights into the interrelationships among different node types.

Both homophily and heterogeneity of graph structures offer a comprehensive reflection of a graph's intrinsic characteristics, enhancing the modeling of complex system behavior. Consequently, the challenge of capturing complete graph structure features lies in effectively capturing and utilizing both types of representations. To address this, we propose a graph representation learning approach that captures both homophily and heteromorphic representations of provenance graphs to improve malicious node detection. The homophily representation uses a self-expression matrix Mo et al. (2024) to aggregate information from nodes of the same type, while the heteromorphic representation employs a heteromorphic encoder to aggregate information from different types of edges surrounding the target node. Unlike previous methods, our approach avoids predefined rules and provides a more detailed characterization of the graph structure, reducing the dependence on high-quality data.

Threat model

This paper presents a methodology for detecting intrusion activities from external sources aimed at acquiring valuable system information. Attackers often execute complex steps to achieve their goals, making it essential to understand the characteristics of these behaviors. We assume that attackers will:

- *Conceal Their Activities*: Attackers mimic normal system operations to remain undetected, blending malicious actions with benign data to mask the intrusion and keep the system functioning normally.
- *Persist Over Time*: Intrusion activities are typically prolonged.

- *Utilize Unknown Vulnerabilities*: Attackers may use zero-day vulnerabilities and other unknown methods, making it difficult to rely on prior knowledge or attack patterns for detection.
- *Leave Identifiable Patterns*: Leave Identifiable Patterns: Despite their efforts, attackers are likely to leave identifiable patterns. For example, there may be notable differences between the local structures of malicious and benign nodes of the same type, as shown in Fig. 1. A process node that suddenly engages in unusual file operations may indicate malicious control and anomalous behavior.

Additionally, we assume attackers cannot alter raw data, such as system audit logs or traffic data, ensuring the integrity and reliability of the constructed provenance graph.

Model design

As shown in Fig. 3, The APT-HERA methodology consists of four main parts: data provenance generator, graph representation module, detection module, and tracing module. The model uses audit logs to build the provenance graph. In the graph representation module, a Multi-Layer Perceptron (MLP) calculates node feature vectors, which are then multiplied with the self-expression matrix to obtain homophily representations. Heteromorphic representations are generated using a heterogeneous encoder, and the two are combined to form the final node embeddings. These embeddings are then input into a lightweight classifier for detection. Finally, the provenance module reconstructs malicious subgraphs based on the identified malicious nodes.

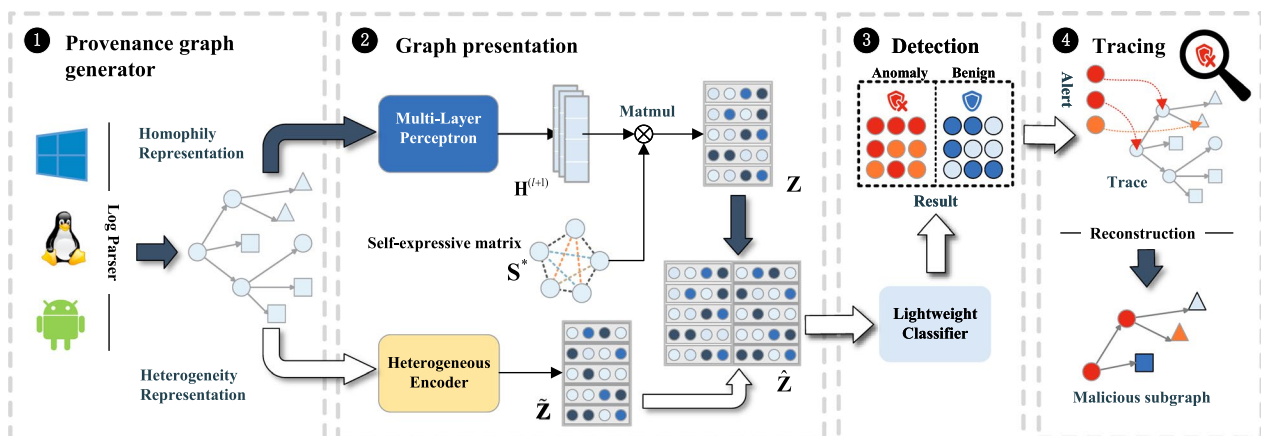


Fig. 3 Overview of APT-HERA's architecture

Provenance graph construction module

Constructing a provenance graph from raw data is essential for graph representation learning and malicious detection. This process involves three key steps: data parsing, feature extraction, and subgraph delineation.

Data parsing

In order to illustrate the data parsing process, we analyse the example of logs. The initial stage of the process is log parsing, whereby system entities and interactions between them are extracted and an initial provenance graph G is constructed. G comprises system entities as nodes N and interactions as edges E . Subsequently, the system entity labels and interaction labels provided in the log entries are utilized as attributes of N and E , respectively. Similarly, when parsing the traffic data, IP addresses are employed as nodes of the provenance graph, the types of interaction protocols between different hosts as edges, and the port numbers opened by the hosts are utilized to represent the node types.

Feature extraction

In the feature extraction phase, the goal is assigning appropriate attributes to nodes and convert them into fixed-size feature vector x . A comparative analysis of the interaction characteristics of benign and malicious nodes reveals significant differences in the number and distribution of interaction types between the two Wang et al. (2022). For example, a malicious scanning program may interact with numerous or frequently the same nodes, resulting in distinct interaction patterns compared to normal process nodes. To capture these interactions, the model counts and encodes the types and distribution of interactions between a target node and its neighbors. Specifically, each node is traversed, its neighboring node types are identified and one-hot encoded, and the cumulative result forms the node's feature vector x . This approach effectively captures both local structural and attribute information, distinguishing between benign and malicious nodes.

Subgraph division

Before graph representation learning, noise reduction and subgraph partitioning operations are performed on G . Redundant edges between pairs of nodes are removed, retaining only one edge. Due to the large scale of the provenance graph, which often consists of hundreds of thousands of nodes and millions of edges, partitioning it into subgraphs is essential to ensure computational efficiency. Each subgraph g contains a fixed number of nodes, which are used for model training.

Specifically, APT-HERA adopts a staged expansion algorithm to construct fixed-size subgraphs. The process begins from a selected seed node. The subgraph is then dynamically expanded using breadth-first search (BFS), progressively including neighboring nodes from the original graph. This expansion continues until the subgraph reaches a predefined node size threshold. This method ensures that all subgraphs have a consistent number of nodes, which facilitates efficient batched training. Moreover, by preserving the local connectivity of the original graph, the approach maintains important structural information within each subgraph.

Graph representation module

The initial node embeddings from provenance graph construction only capture simple interaction states within immediate neighborhoods, which is insufficient for modeling system behavior. To achieve high-quality embeddings, it is crucial to incorporate contextual information such as multi-hop relationships and interactions with other nodes. We employ a graph representation approach to derive high-quality embeddings from feature provenance graphs. This approach consists of two main components: first, capturing the homophily representation of a node within its target neighborhood using a self-expression matrix, and second, obtaining heterogeneity representations through a heterogeneous encoder.

Homophily representation

Extracting homophily representations from the provenance graph refers to using nodes with similar features to linearly represent the target node. The goal is to establish connections between nodes with similar features and aggregate information accordingly. As shown in Fig. 2, we first use the MLP as an encoder to obtain the representation $\mathbf{H}^{(l+1)}$ of the nodes in the $(l+1)$ -th layer:

$$\mathbf{H}^{(l+1)} = \sigma(\mathbf{H}^{(l)}\mathbf{W}^{(l)}). \quad (1)$$

where σ is the activation function, $\mathbf{W}^{(l)}$ denotes the trainable parameters of the encoder, $\mathbf{H}^{(0)}=X$, and X denotes the feature matrix of all nodes in the subspace. In addition, we construct a self-representation matrix $\mathbf{S} \in \mathbb{R}^{N \times N}$ from which each node is linearly represented, an idea similar to the self-attention mechanism Vaswani (2017), as a way to capture the homophily within the subspace:

$$\mathbf{H}^{(l+1)} = \mathbf{S}\mathbf{H}^{(l+1)} + \mathbf{O}^{(l+1)}, \quad (2)$$

where $\mathbf{O}^{(l+1)}$ is the noise matrix. The node representation $h_i^{(l+1)}$ at layer i can be described as $h_i^{(l+1)} = s_{i1}h_1^{(l+1)} + \dots + s_{iN}h_N^{(l+1)}$. If the weight s_{ij} is greater, the probability of node v_i being replaced by node v_j is

greater. To aggregate the information of same-type nodes and efficiently capture the homophily representation of nodes, we want to use as many same-type nodes as possible to represent each other, which requires a larger same-type weight s_{ij} .

In provenance graphs, nodes of the same type often have similar structural properties. For this purpose, we compute the feature distance matrix $\mathbf{D} \in \mathbb{R}^{N \times N}$, where $d_{ij} = \|x_i - x_j\|_2^2$, for all nodes in the subspace, and consider the value less than the threshold as 0 to obtain the sparse feature distance matrix. Then, according to the following method, the homophily is obtained from the subspace and the neighborhood space of the node:

$$\min \left\| \mathbf{H}^{(l+1)} - \mathbf{S} \mathbf{H}^{(l+1)} \right\|_F^2 + \alpha \sum_{i,j=1}^N d_{ij} s_{ij} + \beta \sum_{i,j=1}^N s_{ij}^2, \quad (3)$$

α and β are non-negative parameters used to balance the three terms. In the second term, when d_{ij} is small, s_{ij} becomes larger, indicating that nodes v_i and v_j are similar in character, thus increasing their weight s_{ij} . Conversely, nodes of different types have smaller weights. This ensures that the self-expression matrix focuses on nodes of the same type with similar characteristics while minimizing the influence of nodes of different types. The first term in Equation 3 corresponds to the global reconstruction loss, which captures global connectivity patterns by minimizing the difference between the node features and their reconstructed features. This helps capture connections between distant nodes that are significant for the global structure but may be overlooked in the local structure. The third term regularizes the self-representation matrix $\mathbf{S}$ to prevent trivial solutions. Actually, Equation 3 takes the closed-form solution $\mathbf{S}^*$ as follows:

$$\mathbf{S}^* = \left(\mathbf{H}^{(l+1)} \left(\mathbf{H}^{(l+1)} \right)^T - \frac{\alpha}{2} \mathbf{D} \right) \left(\mathbf{H}^{(l+1)} \left(\mathbf{H}^{(l+1)} \right)^T + \beta \mathbf{I}_N \right)^{-1}, \quad (4)$$

where $\mathbf{I}_N$ is the identity matrix. After achieving $\mathbf{S}^*$, we conduct information aggregation among nodes of the same type in the heterogeneous graph to obtain homophily representations $\mathbf{Z}$:

$$\mathbf{Z} = \mathbf{S}^* \mathbf{H}^{(l+1)}. \quad (5)$$

However, this method causes a large time overhead due to the cubic time complexity in computing $\mathbf{S}^*$ in Equation 4 and the quadratic time complexity in calculating $\mathbf{S}^* \mathbf{H}^{(l+1)}$. Following the analysis in HERO Mo et al. (2024), we compute the homophily representation $\mathbf{Z}$ using matrix identity transformations(details are shown

in Appendix A), which avoids the direct computation of $\mathbf{S}^*$ and reduces the computational overhead:

$$\mathbf{Z} = \mathbf{H}^{(l+1)} \left(\mathbf{H}^{(l+1)} \right)^T \mathbf{B} - \frac{\alpha}{2} \mathbf{D} \mathbf{B}, \quad (6)$$

where

$$\mathbf{B} = \frac{1}{\beta} \mathbf{H}^{(l+1)} - \frac{1}{\beta^2} \mathbf{H}^{(l+1)} \left(\mathbf{I}_d + \frac{1}{\beta} \left(\mathbf{H}^{(l+1)} \right)^T \mathbf{H}^{(l+1)} \right)^{-1} \left(\mathbf{H}^{(l+1)} \right)^T \mathbf{H}^{(l+1)},$$

$\mathbf{I}_d \in \mathbb{R}^{N \times N}$ is the identity matrix, and d is the dimensionality of the node representation. By reordering the matrix multiplication, we further reduce the time complexity of Equation 6 to $\mathcal{O}(Nd^2 + d^3 + kd)$, where $d^2 \ll N$ and $k \ll N^2$, and k indicates the nonzero entries of the $\mathbf{D}$. With this step, we are able to obtain the homophily representation $\mathbf{Z}$ of the subspace of the provenance graph by aggregating the information between nodes of the same type.

Heterogeneity representation

In this phase, the model aggregates information from various types of nodes in the provenance graph. Specifically, a heterogeneous encoder is used for the target node v_i to aggregate the information of its one-hop neighbors, and then the edge-based representation $\tilde{\mathbf{Z}}_{i,r}^{(l+1)}$ is obtained:

$$\tilde{\mathbf{Z}}_{i,r}^{(l+1)} = \delta \left(\left(\frac{1}{m} \sum_{j=1}^m \left\{ \tilde{\mathbf{Z}}_j^{(l)} | v_j \in N_{i,r} \right\} \right) \mathbf{W}_r^{(l)} \right), \quad (7)$$

where δ is the activation function, edge type $r \in R$, $N_{i,r}$ is the set of one-hop neighbors of node v_i with respect to edge r , m is the number of neighbors, and $\mathbf{W}_r^{(l)}$ is the trainable parameter of the heterogeneous encoder. Considering all the edge types in the provenance graph, we further fuse all the edge-based representations to obtain the heterogeneity representation, i.e:

$$\tilde{\mathbf{Z}}^{(l+1)} = \frac{1}{|\mathcal{R}|} \sum_{r \in R} \tilde{\mathbf{Z}}_r^{(l+1)}, \quad (8)$$

where $|\mathcal{R}|$ denotes the number of edge types. The final heterogeneity representation is represented by $\tilde{\mathbf{Z}}$, which incorporates information about all the different types of edges in the subspace and expresses the heterogeneous nature of the provenance graph structure.

Homophily and heterogeneity representations offer distinct views of the same node, each preserving original node characteristics and providing consistent information. Homophily representations aggregate information

from nodes of the same type, whereas heterogeneity representations aggregate information from different node types, highlighting distinct aspects of the target node. To effectively leverage both types of representations, we use a consistency loss $\mathcal{L}_{\text{con}}$ to capture the shared information and a specificity loss $\mathcal{L}_{\text{spe}}$ to maintain unique characteristics. The calculation is as follows:

$$\begin{aligned} L_{\text{con}} = & \mu \log(e^{\sum_{i,j=1}^d \sum_{n=1}^N (p_{ni}p_{nj} + \tilde{p}_{ni}\tilde{p}_{nj})}) \\ & + \sum_{n=1}^N (p_n - \tilde{p}_n)^2, \end{aligned} \quad (9)$$

$$L_{\text{spe}} = \sum_{n=1}^N (q_n - \tilde{q}_n)^2 - \rho \sum_{n=1}^N ((q_n - \tilde{q}_n)^2 + (q_n - p_n)^2). \quad (10)$$

To preserve the distinct information content of homophily and heterogeneity representations, direct alignment is not feasible. Instead, we map these representations into separate potential spaces. Specifically, we map the homophily representation $\mathbf{Z}$ and the heterogeneity representation $\tilde{\mathbf{Z}}$ to a potential space $\mathbf{P}$, where $\mathbf{P} = p_\varphi(\mathbf{Z})$ and $\tilde{\mathbf{P}} = p_\varphi(\tilde{\mathbf{Z}})$. For computing the specificity loss, these representations are mapped to another potential space $\mathbf{Q}$, where $\mathbf{Q} = q_\gamma(\mathbf{Z})$ and $\tilde{\mathbf{Q}} = q_\gamma(\tilde{\mathbf{Z}})$. Here, i and j denote the dimensionality of p_n , and μ and ρ are non-negative parameters. The consistency loss $\mathcal{L}_{\text{con}}$ maximizes the alignment between the homophily and heterogeneity representations, while the specificity loss $\mathcal{L}_{\text{spe}}$ maintains their unique information.

The final objective function is $\mathcal{L} = \mathcal{L}_{\text{con}} + \mathcal{L}_{\text{spe}}$. This function guides the self-supervised learning process of the graph representation module, using $\mathcal{L}$ to provide supervised signals and enable parameter learning in a self-supervised manner. For the subsequent detection task, we combine the homophily and heterogeneity representations by concatenating them horizontally, resulting in the final representation $\hat{\mathbf{Z}}$:

$$\hat{\mathbf{Z}} = (\mathbf{Z}, \tilde{\mathbf{Z}}). \quad (11)$$

As a result, the concatenated representations $\hat{\mathbf{Z}}$ with both homophily and heterogeneity contain more attack related information T than the representations with only homophily $\mathbf{Z}$ or with only heterogeneity $\tilde{\mathbf{Z}}$, i.e.,

$$I(\hat{\mathbf{Z}}, T) \geq \max(I(\mathbf{Z}, T), I(\tilde{\mathbf{Z}}, T)), \quad (12)$$

where $I(\cdot, \cdot)$ indicates the mutual information. Inequality 12 theoretically proves that the concatenated representations representation $\hat{\mathbf{Z}}$ introduces more attack-related information. The detailed proof can be found in Appendix B.

Detection module

Malicious entities typically display structural and attributive features that diverge from the norm. Using conventional classifiers, we can effectively distinguish between benign and malicious node representations. The detection module inputs the final node embeddings from the graph representation phase and identifies anomalous nodes by comparing the predicted node labels with their actual labels.

After evaluating the detection effectiveness and performance of several lightweight classifiers, we select Extreme Gradient Boosting(XGBoost) as our classifier. XGBoost is an efficient gradient boosting framework that improves model performance by iteratively constructing a weak learner, usually a decision tree. Its core principle involves using residuals from the previous round to train a new tree in each step, thereby gradually reducing model error while preventing overfitting through the use of a regularization term. In APT-HERA, the objective function for XGBoost consists of a loss function combined with a regularization term, expressed as:

$$\mathcal{L}(\phi) = \sum_{i=1}^n l(\hat{y}_i, y_i) + \sum_{k=1}^K \Omega(f_k). \quad (13)$$

In this context, $\sum_{i=1}^n l(\hat{y}_i, y_i)$ represents the loss function, which is utilized to quantify the discrepancy between the projected label $\hat{y}_i$ and the actual label y_i of nodes. Additionally, the regularization term, represented by $\Omega(f_k)$, is employed to regulate the complexity of the model. APT-HERA detects anomalous nodes by comparing predicted and actual node label. The model take the final representation $\hat{\mathbf{Z}}$ as input, which considers both homophily and heterogeneity of provenance graph. The excellent performance of the detection module in actual work also reflects the effectiveness of homogeneous and heterogeneity representations for learning malicious behavior patterns.

Tracing module

The tracing module aims to identify and reconstruct malicious subgraphs, enabling security personnel to quickly locate and analyze the source of an attack. In provenance graphs, attack behaviors typically form dense communities Cheng et al. (2024), with the local structure reflecting the execution of intrusion activities. Using community detection algorithms, the module accurately identifies these local structures, maps malicious nodes to subgraphs, and reconstructs the attack process.

Algorithm 1 explains the procedure for reconstructing malicious subgraphs. Initially, the module applies the Louvain Blondel and Guillaume (2008) community detection algorithm to partition the provenance graph

G into multiple communities C . Then, it extracts the subgraph S corresponding to each community. Based on the classifier's detection results, a mapping from malicious nodes to malicious subgraphs is established to reconstruct the initial malicious subgraph. However, the initial malicious subgraph may be too fragmented or redundant, so it is necessary to perform noise reduction and aggregation operations on it.

Firstly, the temporal correlation and potential cross-subgraph interactions between malicious subgraphs are analyzed, with subgraphs exhibiting correlation within a uniform time window aggregated. Secondly, the malicious nodes are used as the centre to filter the boundary nodes that are distant from the subgraphs. This approach effectively connects independently detected malicious nodes, reconstructing the sequence of malicious activities and providing a clear view of their interactions. This helps security personnel quickly identify and understand the attack, reduces analysts' workload, and enhances both system security and management efficiency.

Algorithm 1 Malicious subgraph reconstruction

Input: Provenance graph G , malicious node set $M_{malicious}$
Output: Malicious subgraph set M

```

1: // Step 1: Detect communities using Louvain algorithm
2:  $C \leftarrow \text{Louvain}(G)$ 
3: // Label each community as  $C_i$ 
4: // Step 2: Extract subgraphs from provenance graph
5: for each community  $C_i$  do
6:    $S_i \leftarrow \text{ExtractSubgraph}(G, C_i)$ 
7:   //  $S_i$  is the subgraph corresponding to community  $C_i$ 
8: end for
9: // Step 3: Map malicious nodes to subgraphs
10:  $M \leftarrow \emptyset$ 
11: for each subgraph  $S_i$  do
12:   if exists  $n \in M_{malicious}$  such that  $n \in S_i$  then
13:      $M \leftarrow M \cup S_i$ 
14:     // Add subgraph containing malicious nodes to the set  $M$ 
15:   end if
16: end for
17: // Step 4: Analyze structure and aggregate related malicious subgraphs
18:  $M \leftarrow \text{AnalyzeAndAggregate}(M)$ 
19: return  $M$ 

```

Implementation

We implemented and deployed APT-HERA in lab environment with about 3200 lines of code in Python. We develop several log parsers to collect and process audit logs, including StreamSpot Manzoor et al. (2016) and

Camflow Pasquier et al. (2017). Provenance graphs are constructed using the graph processing library Networkx Hagberg et al. (2008). The graph representation module is implemented via PyTorch Paszke et al. (2019) and DGL Wang et al. (2019). The detection module is developed with Scikit-learn Pedregosa et al. (2011). For parameters of APT-HERA, the learning rate is set to 0.001. We used two hidden layers, with 256 and 512 hidden units in each layer. The final output feature dimension is 16 for node-level detection granularity and 512 for graph-level detection granularity.

Experiments

We evaluated APT-HERA by conducting experiments on four publicly available datasets commonly used for APTs detection to analyze and answer the following questions:

1. *How effective is APT-HERA for the detection of APTs?*
 In Section 6.2.1, we evaluate the effectiveness of APT-HERA in different detection environments, and the experiments show that in APT-HERA, the aver-

age accuracy and false alarm rate in the node-level detection granularity are 99.02% and 0.17%, respectively, which achieves high-precision detection. We conducted a series of comparison experiments between APT-HERA and other state-of-the-art mali-

cious detection methods. The results show that APT-HERA outperforms other methods in node-level detection granularity and has a low time overhead.

2. *How do different components and parameters affect APT-HERA?* We set up ablation experiments in section 6.2.3 to explore the detection effectiveness of different lightweight classifiers, the influence of sub-graph size, the effectiveness of homophily representation and heterogeneity representation, and the role of the tracing module, respectively.

Experiment setup

This section details the experimental setup, datasets, and comparison methods used in the evaluation. The experiments were conducted on a system running Windows 10, with 32 GB of RAM and an A40 GPU (16 GB). The proposed APT-HERA method was compared with five provenance graph-based intrusion detection methods: Unicorn, Log2Vec, Magic, FLASH, and ThreaTrace. Four publicly available datasets were used for the evaluation: DARPA Engagement 3 Keromytis (2018), StreamSpot Manzoor et al. (2016), Unicorn Wget Han et al. (2020), and DARPA 1998 Lincoln Laboratory (1998). All experiments were performed under identical hardware conditions.

Datasets

DARPA Engagement 3 This dataset is sourced from corporate networks during the Cyber Red and Blue Attack and Defense Confrontation, part of the DARPA Transparent Computing Program. The E3 scenario spans two weeks, during which attackers aim to steal proprietary information and personal data from the target company for financial gain. Attack tactics include phishing emails, PowerShell scripts, malicious Excel sheets, Pine backdoors, and malware. Once inside the network, attackers exploit vulnerabilities such as Nginx and Firefox backdoors, Dragon APTs, and Micro APTs to compromise security.

Three subsets of Trace, Theia, and Cadets have been selected as evaluation data for E3. As shown in Table 2, these subsets comprise 51.6 GB of system audit logs,

containing 653,966 system entities and 68,127,444 interaction records.

StreamSpot This dataset is a simulation dataset collected and made publicly available by StreamSpot using the auditing system SystemTap. As illustrated in Table 3, the StreamSpot dataset contains 600 batches of audit logs monitoring system calls under six distinct scenarios. Five of the scenarios emulate benign user behavior, while the attack scenario simulates a download-driven attack. This dataset is regarded as relatively modest in size, and given the absence of labels for log entries and system entities, we undertake graph discrimination detection on the StreamSpot dataset.

Unicorn Wget This dataset includes 150 batches of logs from simulated attacks, with 125 benign batches and 25 containing supply chain attacks that closely resemble normal system activities, making them hard to detect. Due to the large size and complexity of the logs, each provenance graph can contain up to 250,000 nodes, requiring significant memory and potentially affecting detection accuracy. To handle this, we divide the graphs into smaller subgraphs for detection, following a graph-level detection approach similar to other methods.

DARPA 1998 This dataset was created for an intrusion detection evaluation by the U.S. Department of Defense's Advanced Planning Agency at MIT Lincoln Laboratory. It simulates a U.S. Air Force LAN environment, collecting TCPdump network and system audit data over nine weeks, including various user types, network traffic, and attack methods. The dataset offers realistic network conditions but provides less semantic information compared to the E3 dataset, containing only basic details like IP addresses, port numbers, and protocols. For our study, a subset of TCPdump data from two weeks of traffic was used to construct a provenance graph, simulating a scenario with limited semantic data. The node and edge details of the provenance graph are summarized in Table 4.

Table 2 Overview of the DARPA Engagement 3 dataset

Dataset	Scene	# of benign nodes	# of malicious nodes	# of edges
DARPA E3	Trace	2,416,007	67,383	6,978,024
	Theia	3,505,326	25,326	102,929,710
	Cadets	706,966	12,852	8,663,569

Table 3 Overview of StreamSpot and Unicorn Wget datasets

Dataset	scene	# of graphs	Average # of nodes	Average # of edges
StreamSpot	Benign	500	8315	173,857
	Attack	100	8891	28,423
Unicorn Wget	Benign	125	26,524	957,226
	Attack	25	25,716	957,968

Table 4 Overview of DARPA 1998 provenance graph

Dataset	# of benign nodes	# of malicious nodes	# of edges
DARPA 1998	11,347	1582	36,310

Comparison methods

Five provenance graph-based intrusion detection methods were selected for comparison in our evaluation. These methods support both graph-level and node-level detection and are well-suited for comparison across multiple datasets. Notably, ThreaTrace Wang et al. (2022) introduces a novel approach to node-level malicious detection using GraphSAGE to learn node embeddings and identify malicious entities. Flash Rehman et al. (2024) and Magic Jia et al. (2024) are considered state-of-the-art methods. Flash uses Word2Vec to extract semantic features from audit data and a GNN for provenance graph embedding. Magic employs a masked graph autoencoder to reconstruct node features and detect malicious behaviors through outlier detection. The code for all comparison methods was reproduced locally, and experiments were conducted on the same datasets.

Results

We evaluated the effectiveness of APT-HERA's graph-level and node-level APTs detection on four datasets. Here, we show the detection results of APT-HERA on each dataset, and then compare them with state-of-the-art APTs detection methods on these datasets.

APT-HERA's results

The detection results on the four datasets demonstrate that APT-HERA is an effective approach for identifying malicious behaviors across diverse scenarios, achieving a high degree of accuracy while maintaining a low false alarm rate. Table 5 presents the APT-HERA performance metrics in each data set, including precision (Pre), recall (Rec), area under the curve (AUC), F1-score (F1) and false positive rate (FPR).

Table 5 shows that APT-HERA achieves high precision (99.08%), recall (97.68%), and a low false alarm rate (0.16%) in average in node-level detection, effectively distinguishing between benign and malicious behaviors. Notably, in the DARPA E3 test scenario, where the test data is highly imbalanced, APT-HERA maintains a low false alarm rate. This can be attributed to two factors: first, the graph representation module of APT-HERA generates distinct embeddings for benign and malicious entities; second, the XGBoost classifier demonstrates strong performance in handling imbalanced data.

Node-level detection in APT-HERA tends to produce a significant number of false negatives, which often correspond to malicious entities such as malicious files and libraries that are involved in the attack process. This indicates that APT-HERA is proficient at detecting malicious processes and network connections that are distinct from benign entities. However, it has limited capacity to identify passive entities, such as malicious files and libraries. This is due to the fact that the interaction behavior of these passive entities is frequently analogous to that of benign entities. Nevertheless, as long as we can accurately distinguish active entities, such as malicious processes, we can readily identify these potentially intermediate files and libraries during attack traceability.

For graph-level detection, APT-HERA demonstrates strong performance on the simpler StreamSpot dataset. However, its performance on the more complex Unicorn Wget dataset is slightly lower due to the larger size and complexity of the provenance graph. In this context, a small number of malicious features can be obscured by a predominance of benign features in the graph-level embeddings. As shown in Table 5, APT-HERA achieves better results in node-level detection, highlighting that our approach is more effective for entity-level detection. This is because the homophily and heterogeneity representations are designed to capture the structural features and local interactions of nodes. When generalized to graph-level embeddings, the malicious features can be diluted by the many benign features, making graph-level detection more challenging.

Table 5 APT-HERA's detection results

Granularity	Dataset	Pre	Rec	AUC	F1	FPR	TP	FP	TN	FN
graph-level	Streamspot	98.38%	97.40%	99.58%	97.89%	0.33%	64.7	1.1	332.4	1.9
	Unicorn	94.05%	92.34%	99.04%	93.19%	0.84%	25.3	1.6	190	2.1
node-level	E3-Trace	99.10%	99.88%	99.97%	99.49%	0.44%	53458	484	109291	65
	E3-Theia	98.37%	93.25%	99.93%	95.74%	0.15%	19125	317	214296	1384
	E3-Cadets	99.21%	99.58%	99.98%	99.39%	0.03%	10793	86	290695	46
	DARPA-1998	98.39%	97.99%	99.96%	98.19%	0.26%	1223	20	7747	25

Comparison experiment

In comparison experiments, we compare five provenance graph-based APTs detection methods using the same dataset. For example, Unicorn Han et al. (2020), a graph-level detection method using the StreamSpot and Unicorn Wget datasets, Log2Vec Liu et al. (2019), ThreaTrace Wang et al. (2022), Flash Rehman et al. (2024), and Magic Jia et al. (2024) using the DARPA E3 dataset, and also they implement different granularities of detection, which is comparable to our approach.

Table 6 compares APT-HERA with other methods across different datasets. The best-performing result for each metric is highlighted in bold. APT-HERA exhibits exceptional stability in node-level detection, achieving the lowest false positive rates in two out of four node-level detection while maintaining competitive precision. Notably, in resource-constrained environments, our method demonstrates balanced performance with 98% precision and 0.26% FPR, significantly outperforming

counterparts in reliability. The closest competitor is Magic, which uses extensive benign data to learn normal behavioral patterns and identifies malicious nodes through outlier detection. While Magic achieves good results, its reliance on learning numerous benign patterns significantly increases its training and inference time. In contrast, APT-HERA does not require pre-learning of normal behavior patterns. As shown in Fig. 4, Magic's inference time is 5–8 times longer than that of APT-HERA, demonstrating APT-HERA's superior speed and efficiency. For example, the Cadets subset of DARPA E3 collected 17.91 GB of audit logs over two weeks, averaging 1.27 GB per day. Using APT-HERA, a day's data can be processed in 1–2 min after preprocessing.

While our approach improves detection speed, it also increases memory overhead. This is because constructing the self-expression matrix to learn the homophily representation of a node requires computing feature distances between nodes in the subspace, consuming significant memory. However, memory consumption can be reduced by adjusting the subgraph size used as model inputs, which will be further analyzed in Section 6.2.3.

We simulated a detection environment with insufficient high-quality data by constructing a provenance graph using the DARPA 1998 dataset. In this scenario, our method demonstrates superior detection results relative to other techniques. This illustrates that APT-HERA is capable of effectively extracting the structural information embedded in the provenance graph, enabling it to adapt effectively to scenarios with limited information. In comparison, Flash exhibits the lowest performance in this dataset. This can be attributed to the fact that Flash relies primarily on the Word2Vec method to learn word embedding representations from audit logs, with the aim of obtaining rich semantic information.

Table 6 Comparison experiment results

Dataset	Approach	Precision	Recall	F1-Score	FPR
StreamSpot	Unicorn	95%	93%	96%	1.60%
	ThreaTrace	98%	99%	99%	0.40%
	Flash	100%	96%	98%	0.40%
	Magic	99%	100%	99%	0.60%
	Ours	98%	97%	97%	0.33%
Unicorn Wget	Unicorn	86%	95%	90%	15.50%
	ThreaTrace	91%	96%	93%	7.40%
	Flash	98%	96%	94%	0.10%
	Magic	98%	96%	97%	2.00%
	Ours	94%	92%	93%	0.84%
DARPA E3 Trace	Log2Vec	54%	78%	64%	1.80%
	ThreaTrace	72%	99%	83%	1.10%
	Flash	95%	99%	97%	0.10%
	Magic	99%	99%	99%	0.10%
	Ours	99%	99%	99%	0.44%
DARPA E3 Theia	Log2Vec	62%	66%	64%	0.30%
	ThreaTrace	87%	99%	93%	0.10%
	Flash	92%	99%	95%	0.06%
	Magic	98%	99%	99%	0.10%
	Ours	98%	93%	96%	0.15%
DARPA E3 Cadets	Log2Vec	49%	85%	62%	1.60%
	ThreaTrace	92%	99%	95%	0.20%
	Flash	93%	99%	96%	0.10%
	Magic	94%	99%	97%	0.20%
	Ours	99%	99%	99%	0.03%
DARPA 1998	ThreaTrace	89%	99%	94%	0.78%
	Flash	84%	94%	88%	6.80%
	Magic	95%	98%	96%	0.57%
	Ours	98%	98%	98%	0.26%

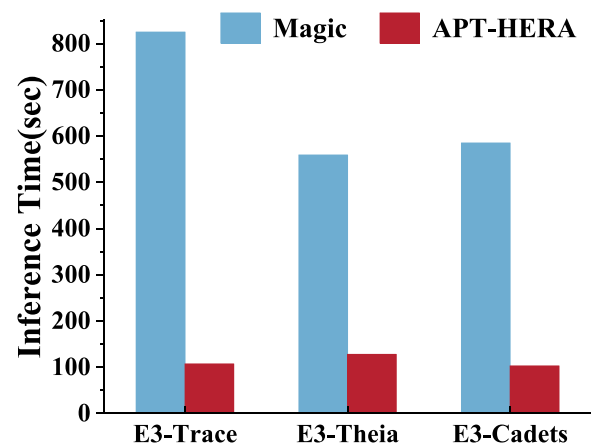


Fig. 4 Inference time comparison between APT-HERA and Magic

However, in scenarios where information is limited, this approach tends to result in significantly impaired detection outcomes.

Ablation study

In the ablation study, we conducted a comprehensive evaluation of the performance of APT-HERA by varying various parameter settings and components to investigate their influence on the model's effectiveness. The key components that we investigated include the impact of different lightweight classifiers, the effect of subgraph size on detection effectiveness, the effectiveness of homophily and heterogeneity representations, and the role of the attack tracing module.

Lightweight classifiers We evaluated the performance of several lightweight classifiers on the DARPA E3 Cadets dataset, including XGBoost (XGB), Support Vector Machine (SVM), Random Forest (RF), Logistic Regression (LR), and Decision Tree (DT). SVM are supervised learning models that classify data points by finding optimal hyperplanes. Random Forest improves classification accuracy and reduces overfitting by constructing multiple decision trees and combining their outputs through voting. Logistic Regression is a statistical model for binary classification that estimates the probability of a binomial outcome using a linear regression approach. Decision Trees are tree-structured models that perform classification or regression through iterative feature partitioning.

Figure 5 shows the mean detection performance of these classifiers on the Cadets dataset, while Fig. 6 illustrates their time overhead. XGBoost achieves similar detection performance to SVM and Random Forest but with significantly lower time overhead. Logistic Regression, while fast, has lower detection accuracy compared to XGBoost. Overall, XGBoost demonstrates a clear advantage in balancing detection performance with computational efficiency. This is consistent with findings

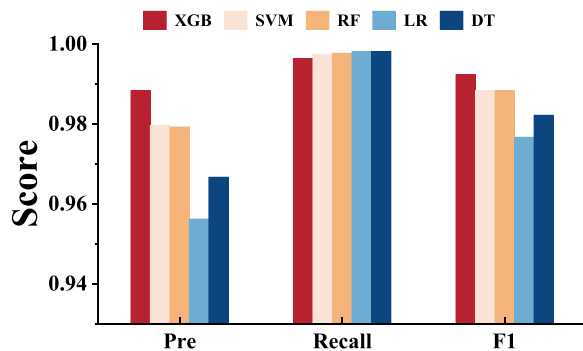


Fig. 5 Performance of different lightweight classifiers

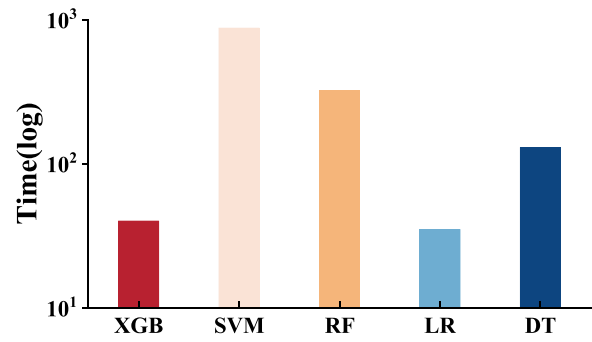


Fig. 6 Time overhead of different lightweight classifiers

from other studies highlighting XGBoost's superior performance Anghel et al. (2018); Si et al. (2017).

Influence of subgraph size APT-HERA partitions the original provenance graph into smaller subgraphs. The size of these subgraphs affects both homophily and heterogeneity representations, which capture a node's local interaction features. This section examines how varying subgraph sizes impact detection performance, time overhead, and memory consumption using the Cadets dataset.

As shown in Fig. 7, with subgraph sizes under 7000 nodes, the model maintains high accuracy, recall, and F1 scores. However, as the number of nodes in the subgraph increases, the accuracy and F1 score tend to decline. This decline is due to the increased presence of irrelevant nodes, which can negatively impact the process of generating homophily and heterogeneity representations.

Fig. 8 shows the relationship between program runtime and subgraph size. The time overhead decreases when the number of nodes in the subgraph is between 4,000 and 7,000. Conversely, runtime increases for subgraphs that are either too small or too large. This behavior can be explained by analyzing the model's time complexity. Let N be the total number of nodes in the provenance graph and n be the number of nodes in the subgraph ($n < N$). Let d denote the dimension of the node embedding.

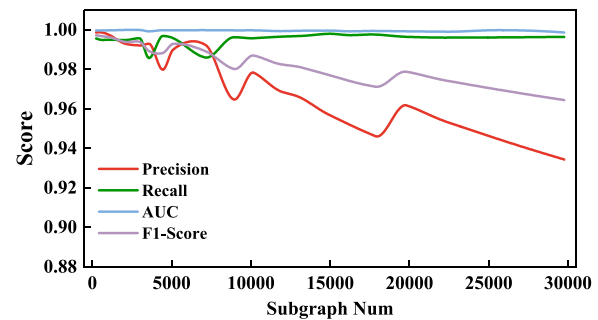


Fig. 7 Effect of subgraph size on system performance

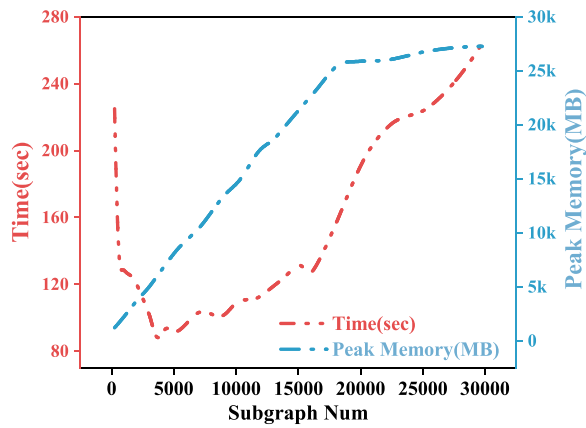


Fig. 8 Effect of subgraph size on time and memory overhead

The time complexity of computing the feature distance in the subgraph within the graph representation module is $\mathcal{O}(n^2)$. In Equation 6, the time complexity of computing a matrix transpose is $\mathcal{O}(d^3)$, the time complexity of computing $\mathbf{H}^{(l+1)}(\mathbf{H}^{(l+1)})^{TB}$ is $\mathcal{O}(nd^2)$, and the time complexity of computing $\frac{\alpha}{2}DB$ is $\mathcal{O}(kd)$, where k denotes the non-zero term in the feature distance. Thus, the total time complexity of the model is: $\mathcal{O}(Nd^2 + (Nd^3)/n + Nkd/n + Nn)$. It is seen that the second term increases rapidly when the number of subgraph nodes n is small and continues to decline. As n increases, the second term decreases rapidly and levels off, while the fourth term leads to a continuous increase in the model time overhead, which is consistent with the results shown in Fig. 8.

Equations 6 and 9 respectively have space complexities of $\mathcal{O}(nd + d^2)$ and $\mathcal{O}(|\mathcal{R}| \cdot nd)$, where $d \ll n$ and $|\mathcal{R}| \gg 1$. Thus, the overall space complexity of APT-HERA can be approximated as $\mathcal{O}(|\mathcal{R}| \cdot Nd)$, dominated by the number of relations and nodes. As illustrated in Fig. 8, the peak memory usage increases approximately linearly with subgraph size. For instance, when the subgraph contains 1,000 nodes, the memory peak reaches approximately 2,000 MB.

In practice, subgraph size can be adjusted based on deployment scenarios to balance performance and efficiency. For low-resource environments (e.g., edge devices), smaller subgraphs keep memory under 3000MB and inference time below 100 s while maintaining good detection performance. For high-performance settings (e.g., server clusters), larger subgraphs can be used with GPU acceleration and distributed computing to maximize performance. However, overly large subgraphs may introduce performance fluctuations due to feature sparsity, which can be mitigated through dynamic pruning or adaptive subgraph partitioning.

Effectiveness of homophily and heterogeneity representations: This study assessed the impact of homophily and heterogeneity representations on the detection performance of APT-HERA using the DARPA E3 dataset. Fig. 9 shows the results for detection using only homophily representation (Hom), only heterogeneity representation (Het), and a combination of both (Com). The analysis reveals that while each representation type alone can provide good detection results, their performance can be unstable and vary significantly across different datasets. However, combining both representations results in more consistent and stable detection performance across various datasets.

The function of the attack tracing module: The role of the attack tracing module in APT-HERA is pivotal for identifying and managing APTs. Traditional provenance graph analysis often involves time-consuming and expertise-intensive processes for anomaly detection and attack reconstruction. In contrast, APT-HERA leverages community detection algorithms to identify malicious subgraphs, which reflect the intrusion process of malicious entities. This approach streamlines alert verification and security management by clustering related malicious entities, as shown in Fig. 10. By linking these entities based on their relationships, APT-HERA significantly reduces the number of alerts that security personnel must address, similar to the method used in KAIROS Cheng et al. (2024).

Discussion and limitations

Memory Overhead: The reliance on the feature distance matrix for obtaining embedding representations in APT-HERA significantly increases memory consumption. Although adjusting subgraph size can mitigate this issue, certain scenarios may necessitate higher memory usage to achieve improved detection results. To mitigate the memory overhead caused by reliance on the feature distance matrix in APT-HERA, several optimizations can be considered: apply feature selection or dimensionality reduction techniques to eliminate noise and redundant information, thereby simplifying the matrix; replace full distance matrices with sparse or low-rank approximations; or compute similarities based on local neighborhoods instead of global pairwise distances to reduce computational complexity.

Attack Reconstruction: Although APT-HERA facilitates the recovery of malicious subgraphs, it has limitations. The current approach links individual alerts and node-to-graph mappings but does not fully address the complex causal relationships between malicious entities, potentially leading to incomplete feedback on the entire attack flow. Future work will focus on developing attack reconstruction methods that better align with our model,

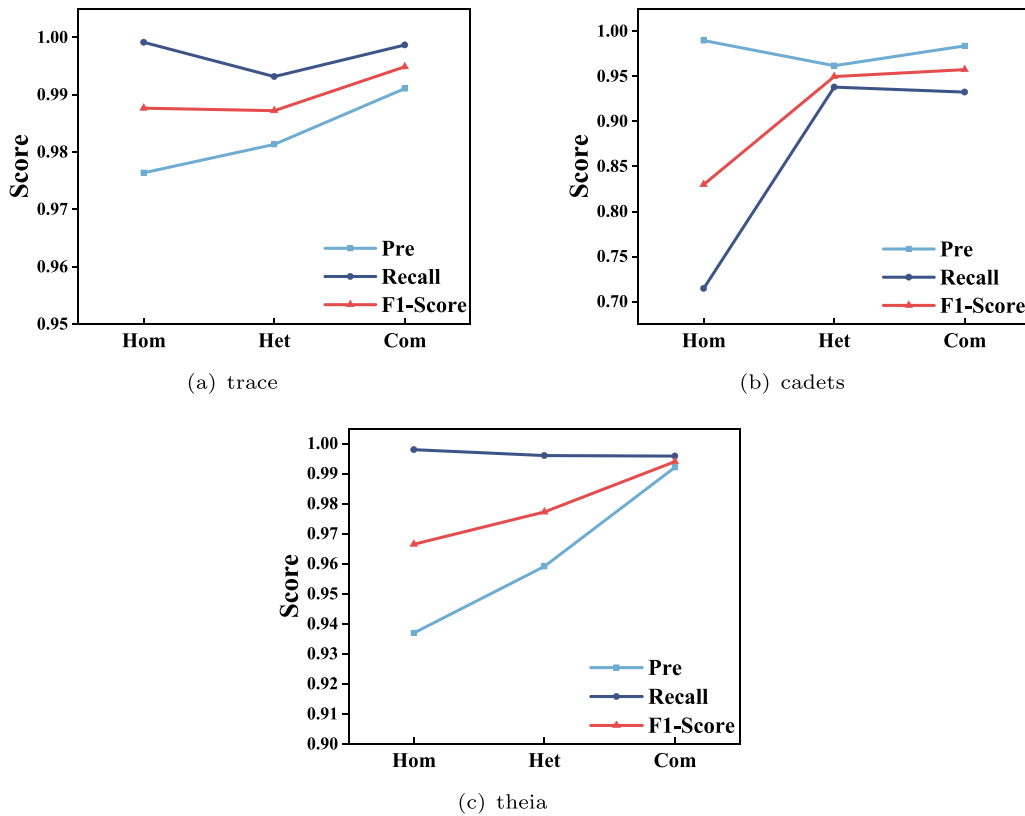


Fig. 9 Effect of homophily and heterogeneity representation on detection results

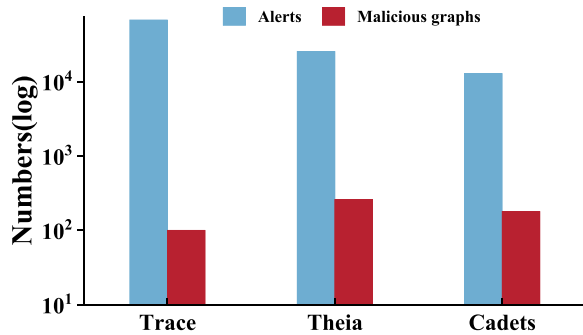


Fig. 10 Comparison of the number of original alerts and generated malicious subgraphs

incorporating causal links and attack flow to provide a comprehensive reconstruction of the attack narrative.

Conceptual Drift: Conceptual drift, which refers to the evolution of data distribution in the underlying system over time, poses a significant challenge for intrusion detection models. As new system activities emerge, previously learned patterns may become obsolete, leading to potential misclassifications. While APT-HERA's minimal time overhead allows for adaptation to new environments

through periodic retraining, this does not fully resolve the issue of concept drift. Addressing this challenge effectively remains an area for future research.

Reliance on Log Integrity: In this work, we make an idealized assumption that the attacker cannot modify the log content. However, we acknowledge that this assumption may not hold in real-world scenarios. As a potential direction for future work, a Graph Autoencoder (GAE) could be employed to detect anomalous log modification patterns. By training the GAE on subgraphs derived from legitimate behavior, subgraphs exhibiting higher reconstruction errors may indicate possible tampering.

Conclusion

In this paper we design and implement APT-HERA, which is able to adapt to detection environments with insufficient high-quality data and achieve highly accurate attack detection and traceability with low time overhead. APT-HERA uses heterogeneous graph representation learning to learn rich structural features from raw audit logs, acquires homophily and heterogeneity representations of provenance graphs to learn the behavioral patterns of the system's activities, and utilizes a lightweight classifier to perform APT detection with adjustable

granularity. The experimental evaluation results demonstrate that APT-HERA achieves high accuracy and a low false alarm rate in a variety of detection scenarios.

Appendix A Matrix transformation

Given four matrices, $\mathbf{A} \in \mathbb{R}^{n \times n}$, $\mathbf{U} \in \mathbb{R}^{n \times k}$, $\mathbf{C} \in \mathbb{R}^{k \times k}$, and $\mathbf{V} \in \mathbb{R}^{k \times n}$, where n, k are dimensions of these matrices, according to the Woodbury identity matrix transformation Woodbury (1950), we have

$$(\mathbf{A} + \mathbf{UCV})^{-1} = \mathbf{A}^{-1} - \mathbf{A}^{-1}\mathbf{U}(\mathbf{C}^{-1} + \mathbf{VA}^{-1}\mathbf{U})^{-1}\mathbf{VA}^{-1}. \quad (\text{A1})$$

Without loss of generality, the matrices $\mathbf{A}$ and $\mathbf{C}$ can be replaced with the identity matrix; therefore, we further have

$$(\mathbf{I} + \mathbf{UV})^{-1} = \mathbf{I} - \mathbf{U}(\mathbf{I} + \mathbf{VU})^{-1}\mathbf{V}. \quad (\text{A2})$$

Based on Equation A2, we can transform $(\mathbf{H}^{(l+1)}(\mathbf{H}^{(l+1)})^T + \beta\mathbf{I}_N)^{-1}$ in Equation 4 as:

$$\begin{aligned} & (\mathbf{H}^{(l+1)}(\mathbf{H}^{(l+1)})^T + \beta\mathbf{I}_N)^{-1} \\ &= \frac{1}{\beta}\mathbf{I} - \frac{1}{\beta^2}\mathbf{H}^{(l+1)}\left(\mathbf{I}_d + \frac{1}{\beta}(\mathbf{H}^{(l+1)})^T\mathbf{H}^{(l+1)}\right)^{-1}(\mathbf{H}^{(l+1)})^T. \end{aligned} \quad (\text{A3})$$

Therefore, with Equation 4 and Equation A3, we can transform Equation 5 as

$$\begin{aligned} \mathbf{z} &= \mathbf{S}^*\mathbf{H}^{(l+1)} \\ &= \left(\mathbf{H}^{(l+1)}(\mathbf{H}^{(l+1)})^T - \frac{\alpha}{2}\mathbf{D}\right)\left(\mathbf{H}^{(l+1)}(\mathbf{H}^{(l+1)})^T + \beta\mathbf{I}_N\right)^{-1}\mathbf{H}^{(l+1)} \\ &= \left(\mathbf{H}^{(l+1)}(\mathbf{H}^{(l+1)})^T - \frac{\alpha}{2}\mathbf{D}\right) \\ &\quad \left(\frac{1}{\beta}\mathbf{I} - \frac{1}{\beta^2}\mathbf{H}^{(l+1)}\left(\mathbf{I}_d + \frac{1}{\beta}(\mathbf{H}^{(l+1)})^T\mathbf{H}^{(l+1)}\right)^{-1}(\mathbf{H}^{(l+1)})^T\right)\mathbf{H}^{(l+1)}. \end{aligned} \quad (\text{A4})$$

Then we replace $\left(\frac{1}{\beta}\mathbf{I} - \frac{1}{\beta^2}\mathbf{H}^{(l+1)}\left(\mathbf{I}_d + \frac{1}{\beta}(\mathbf{H}^{(l+1)})^T\mathbf{H}^{(l+1)}\right)^{-1}(\mathbf{H}^{(l+1)})^T\right)\mathbf{H}^{(l+1)}$ with $\mathbf{B}$ and obtain

$$\begin{aligned} \mathbf{Z} &= \left(\mathbf{H}^{(l+1)}(\mathbf{H}^{(l+1)})^T - \frac{\alpha}{2}\mathbf{D}\right)\mathbf{B} \\ &= \mathbf{H}^{(l+1)}(\mathbf{H}^{(l+1)})^T\mathbf{B} - \frac{\alpha}{2}\mathbf{DB}. \end{aligned} \quad (\text{A5})$$

This is exactly the Equation 6

Appendix B Proof of inequality 12

In the following proofs, for random variables A, B, C , we use $I(A, B)$ to represent the mutual information between A and B , and we use $I(A, B | C)$ to represent conditional mutual information of A and B on a given C , use $H(A)$ for the entropy, and $H(A | B)$ for the conditional entropy. We first list some properties of mutual information and entropy that will be used in the proofs.

- *Property 1.* Relationship between the mutual information and entropy:

$$I(A, B) = H(A) - H(A | B). \quad (\text{B6})$$

- *Property 2.* Relationship between the conditional mutual information and entropy:

$$I(A, B | C) = H(A | C) - H(A | B, C). \quad (\text{B7})$$

- *Property 3.* Non-negativity of mutual information:

$$I(A, B) \geq 0, \quad I(A, B | C) \geq 0. \quad (\text{B8})$$

- *Property 4.* Relationship between the conditional entropy and entropy:

$$H(A | B) = H(A, B) - H(B). \quad (\text{B9})$$

Proof

Given the fused representations $\hat{\mathbf{Z}}$ that contain both homophily and heterogeneity, we have

$$H(\hat{\mathbf{Z}}) = H(\mathbf{Z}|\tilde{\mathbf{Z}}) + H(\tilde{\mathbf{Z}}|\mathbf{Z}) + I(\tilde{\mathbf{Z}}, \mathbf{Z}), \quad (\text{B10})$$

where $H(\mathbf{Z}|\tilde{\mathbf{Z}})$ and $H(\tilde{\mathbf{Z}}|\mathbf{Z})$ indicate the specific information of $\mathbf{Z}$ and $\tilde{\mathbf{Z}}$ respectively, and $I(\tilde{\mathbf{Z}}, \mathbf{Z})$ indicates the consistent information between $\mathbf{Z}$ and $\tilde{\mathbf{Z}}$. According to Properties 1 and 4, we have

$$\begin{aligned} H(\hat{\mathbf{Z}}) &= H(\mathbf{Z}|\tilde{\mathbf{Z}}) + H(\tilde{\mathbf{Z}}|\mathbf{Z}) + I(\tilde{\mathbf{Z}}, \mathbf{Z}) \\ &= H(\mathbf{Z}|\tilde{\mathbf{Z}}) + H(\tilde{\mathbf{Z}}|\mathbf{Z}) + H(\mathbf{Z}) - H(\mathbf{Z}|\tilde{\mathbf{Z}}) \\ &= H(\mathbf{Z}|\tilde{\mathbf{Z}}) + H(\tilde{\mathbf{Z}}|\mathbf{Z}) + H(\mathbf{Z}, \tilde{\mathbf{Z}}) - H(\tilde{\mathbf{Z}}|\mathbf{Z}) - H(\mathbf{Z}|\tilde{\mathbf{Z}}) \\ &= H(\mathbf{Z}, \tilde{\mathbf{Z}}). \end{aligned} \quad (\text{B11})$$

Therefore, for any downstream task T , we further have

$$H(\hat{\mathbf{Z}}, T) = H(\mathbf{Z}, \tilde{\mathbf{Z}}, T). \quad (\text{B12})$$

To prove $I(\hat{\mathbf{Z}}, T) \geq \max(I(\mathbf{Z}, T), I(\tilde{\mathbf{Z}}, T))$, we first prove $I(\hat{\mathbf{Z}}, T) \geq I(\mathbf{Z}, T)$. Then based on Eq.B11, Eq.B12, Property 1 and Property 4, we can transform $I(\hat{\mathbf{Z}}, T)$ as follows:

$$\begin{aligned}
I(\hat{\mathbf{Z}}, T) &= H(\hat{\mathbf{Z}}) - H(\hat{\mathbf{Z}}|T) \\
&= H(\hat{\mathbf{Z}}) - H(\hat{\mathbf{Z}}, T) + H(T) \\
&= H(\mathbf{Z}, \tilde{\mathbf{Z}}) - H(\mathbf{Z}, \tilde{\mathbf{Z}}, T) + H(T).
\end{aligned} \tag{B13}$$

Moreover, based on Properties 1 and 2, we have

$$I(\mathbf{Z}, T) = H(\mathbf{Z}) - H(\mathbf{Z}|T), \tag{B14}$$

$$\begin{aligned}
I(\mathbf{Z}, T|\mathbf{Z}) &= H(\mathbf{Z}|\mathbf{Z}) - H(\mathbf{Z}|\mathbf{Z}, T) \\
&= H(\mathbf{Z}, \tilde{\mathbf{Z}}) - H(\mathbf{Z}) - H(\tilde{\mathbf{Z}}|\mathbf{Z}, T).
\end{aligned} \tag{B15}$$

Then with Eq.B14, Eq.B15 and Property 4, we can obtain

$$\begin{aligned}
I(\mathbf{Z}, T) + I(\tilde{\mathbf{Z}}, T|\mathbf{Z}) &= H(\mathbf{Z}) - H(\mathbf{Z}|T) + H(\mathbf{Z}, \tilde{\mathbf{Z}}) - H(\mathbf{Z}) - H(\tilde{\mathbf{Z}}|\mathbf{Z}, T) \\
&= H(\mathbf{Z}, \tilde{\mathbf{Z}}) - H(\mathbf{Z}|T) - H(\tilde{\mathbf{Z}}|\mathbf{Z}, T) \\
&= H(\mathbf{Z}, \tilde{\mathbf{Z}}) - H(\mathbf{Z}, T) + H(T) - H(\tilde{\mathbf{Z}}|\mathbf{Z}, T) \\
&= H(\mathbf{Z}, \tilde{\mathbf{Z}}) - H(\mathbf{Z}, T) + H(T) - H(\tilde{\mathbf{Z}}, \mathbf{Z}, T) + H(\mathbf{Z}, T) \\
&= H(\mathbf{Z}, \tilde{\mathbf{Z}}) + H(T) - H(\tilde{\mathbf{Z}}, \mathbf{Z}, T).
\end{aligned} \tag{B16}$$

According to Eq.B13 and Eq.B16, we have

$$I(\hat{\mathbf{Z}}, T) = I(\mathbf{Z}, T) + I(\tilde{\mathbf{Z}}, T|\mathbf{Z}). \tag{B17}$$

Based on Property 3, we have $I(\tilde{\mathbf{Z}}, T|\mathbf{Z}) \geq 0$, so we can get

$$I(\hat{\mathbf{Z}}, T) \geq I(\mathbf{Z}, T). \tag{B18}$$

Similarly, we can also obtain

$$I(\hat{\mathbf{Z}}, T) \geq I(\tilde{\mathbf{Z}}, T). \tag{B19}$$

Therefore, $I(\hat{\mathbf{Z}}, T) \geq \max(I(\mathbf{Z}, T), I(\tilde{\mathbf{Z}}, T))$ and we complete the proof. $\square$

Acknowledgements

This work is supported partially by the National Natural Science Foundation of China [61972096, 61771140, 61872088, 61872090, 61902289], the University-Industry Cooperation of Fujian Province [2022H6025].

Author contributions

Yuanhuang Liu: Conceptualization, Investigation, Methodology, Software, Project administration, Validation, Writing original draft. Ayong Ye: Funding acquisition, Methodology. Wenting Lu: Validation and Investigation. Longjing Yang: Validation and Investigation.

Funding

This work is supported partially by the National Natural Science Foundation of China [61972096, 61771140, 61872088, 61872090, 61902289], the University-Industry Cooperation of Fujian Province [2022H6025].

Data availability

Data will be made available on request.

Declarations

Conflict of interest

The authors declare that they have no Conflict of interest.

Received: 10 March 2025 Accepted: 11 June 2025

Published online: 05 March 2026

References

- Alsaheel A, Nan Y, Ma S, Yu L, Walkup G, Celik ZB, Zhang X, Xu D (2021) ATLAS: a sequence-based learning approach for attack investigation. In: 30th USENIX security symposium (USENIX Security 21), pp 3005–3022. USENIX Association, ??? <https://www.usenix.org/conference/usenixsecurity21/presentation/alsaheel>
- Aly A, Iqbal S, Youssef A, Mansour E (2024) Megr-apt: a memory-efficient apt hunting system based on attack representation learning. *IEEE Trans Inf Forensics Secur*. <https://doi.org/10.1109/TIFS.2024.3396390>
- Anghel A, Papandreu N, Parnell T, De Palma A, Pozidis H (2018) Benchmarking and optimization of gradient boosting decision tree algorithms. *arXiv:1809.04559*<https://doi.org/10.48550/arXiv.1809.04559>
- Blondel VD, Guillaume J-L, Lambiotte R, Lefebvre E (2008) Fast unfolding of communities in large networks. *J Stat Mech Theory Exp* 2008(10):10008. <https://doi.org/10.1088/1742-5468/2008/10/P10008>
- Bolton AD, Anderson-Cook CM (2017) Apt malware static trace analysis through bigrams and graph edit distance. *Stat Anal Data Min ASA Data Sci J* 10(3):182–193
- Center for Internet Security. (2021) The solarwinds cyber-attack: what you need to know. Available from <https://www.cisecurity.org/solarwinds>. Accessed 18 Sept 2024
- Chen T, Dong C, Lv M, Song Q, Liu H, Zhu T, Xu K, Chen L, Ji S, Fan Y (2022) Apt-kgl: an intelligent apt detection system based on threat knowledge and heterogeneous provenance graph learning. *IEEE Trans Dependable Secure Comput*. <https://doi.org/10.1109/TDSC.2022.3229472>
- Cheng Z, Lv Q, Liang J, Wang Y, Sun D, Pasquier T, Han X (2024) Kairos: practical intrusion detection and investigation using whole-system provenance, pp 3533–3551 <https://doi.org/10.1109/SP54263.2024.00005>
- Church KW (2017) Word2vec. *Nat Lang Eng* 23(1):155–162. <https://doi.org/10.1017/S1351324916000334>
- Ding H, Zhai J, Nan Y, Ma S (2023) AIRTAG: towards automated attack investigation by unsupervised learning with log texts. In: 32nd USENIX security symposium (USENIX Security 23), pp 373–390. USENIX Association, Anaheim. <https://www.usenix.org/conference/usenixsecurity23/presentation/ding-hailun-airtag>
- Du M, Li F, Zheng G, Srikumar V (2017) Deeplog: anomaly detection and diagnosis from system logs through deep learning. In: Proceedings of the 2017 ACM SIGSAC conference on computer and communications security, pp 1285–1298. <https://doi.org/10.1145/3133956.3134015>
- Fang Y, Wang C, Fang Z, Huang C (2022) Lmtracker: Lateral movement path detection based on heterogeneous graph embedding. *Neurocomputing* 474:37–47. <https://doi.org/10.1016/j.neucom.2021.12.026>
- Grover A, Leskovec J (2016) node2vec: scalable feature learning for networks. In: KDD '16, pp 855–864. Association for Computing Machinery, New York. <https://doi.org/10.1145/2939672.2939754>
- Hagberg A, Swart PJ, Schult DA (2008) Exploring network structure, dynamics, and function using networkx. In: Technical report, Los Alamos National Laboratory (LANL), Los Alamos, NM (United States)
- Hamilton W, Ying Z, Leskovec J (2017) Inductive representation learning on large graphs. *Advances in neural information processing systems*. 30
- Han X, Pasquier T, Bates A, Mickens J, Seltzer M (2020) Unicorn: runtime provenance-based detector for advanced persistent threats. <https://doi.org/10.14722/ndss.2020.24046>, [arXiv:2001.01525](https://arxiv.org/abs/2001.01525)
- Hassan WU, Guo S, Li D, Chen Z, Jee K, Li Z, Bates A (2019) Nodozo: combatting threat alert fatigue with automated provenance triage. In: Network and distributed systems security symposium
- Hassan WU, Bates A, Marino D (2020) Tactical provenance analysis for endpoint detection and response systems. In: 2020 IEEE symposium on security

- and privacy (SP), IEEE, pp 1172–1189. <https://doi.org/10.1109/SP40000.2020.00096>
- Hossain MN, Milajerdi SM, Wang J, Eshete B, Gjomemo R, Sekar R, Stoller S, Venkatakrishnan VN (2017) SLEUTH: real-time attack scenario reconstruction from cots audit data. In: 26th USENIX security symposium (USENIX Security 17), pp 487–504. USENIX Association, Vancouver, BC. <https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/hossain>
- Hossain MN, Sheikhi S, Sekar R (2020) Combating dependence explosion in forensic analysis using alternative tag propagation semantics. In: 2020 IEEE symposium on security and privacy (SP), IEEE, pp 1139–1155. <https://doi.org/10.1109/SP40000.2020.00064>
- Jia Z, Xiong Y, Nan Y, Zhang Y, Zhao J, Wen M (2024) MAGIC: detecting advanced persistent threats via masked graph representation learning. In: 33rd USENIX security symposium (USENIX Security 24), pp 5197–5214. USENIX Association, Philadelphia, PA. <https://www.usenix.org/conference/usenixsecurity24/presentation/jia-zian>
- Keromytis AD (2018) Transparent Computing Engagement 3 data release. Available from <https://github.com/darpa-i2o/Transparent-Computing/blob/master/README-E3.md>. Accessed 18 Sept 2024
- Lai S, Xu L, Liu K, Zhao J (2015) Recurrent convolutional neural networks for text classification. In: Proceedings of the AAAI conference on artificial intelligence, vol 29. <https://doi.org/10.1609/aaai.v29i1.9513>
- Laurenza G, Aniello L, Lazzaretto R, Baldoni R (2017) Malware triage based on static features and public apt reports. In: cyber security cryptography and machine learning: first international conference, CSCML 2017, Beer-Sheva, Israel, June 29–30, 2017, Proceedings 1, Springer, pp 288–305
- Li Z, Chen QA, Yang R, Chen Y, Ruan W (2021) Threat detection and investigation with system-level provenance graphs: a survey. *Comput Secur* 106:102282. <https://doi.org/10.1016/j.cose.2021.102282>
- Li T, Jiang Y, Lin C, Obaidat MS, Shen Y, Ma J (2022) Deepag: attack graph construction and threats prediction with bi-directional deep learning. *IEEE Trans Dependable Secure Comput* 20(1):740–757
- Lincoln Laboratory (1998) 1998 DARPA intrusion detection evaluation dataset. Available from <https://www.ll.mit.edu/r-d/datasets/1998-darpa-intrusion-detection-evaluation-dataset>. Accessed 18 Sept 2024
- Liu F, Wen Y, Zhang D, Jiang X, Xing X, Meng D (2019) Log2vec: a heterogeneous graph embedding based approach for detecting cyber threats within enterprise. In: Proceedings of the 2019 ACM SIGSAC conference on computer and communications security, pp 1777–1794. <https://doi.org/10.1145/3319535.3363224>
- Liu Y, Zhang M, Li D, Jee K, Li Z, Wu Z, Rhee J, Mittal P (2018) Towards a timely causality analysis for enterprise security. In: NDSS
- Manzoor E, Milajerdi SM, Akoglu L (2016) Fast memory-efficient anomaly detection in streaming heterogeneous graphs. In: Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining, pp 1035–1044. <https://doi.org/10.1145/2939672.2939783>
- Milajerdi SM, Eshete B, Gjomemo R, Venkatakrishnan V (2019) Piroit: aligning attack behavior with kernel audit records for cyber threat hunting. In: Proceedings of the 2019 ACM SIGSAC conference on computer and communications security, pp 1795–1812. <https://doi.org/10.1145/3319535.3363217>
- Milajerdi SM, Gjomemo R, Eshete B, Sekar R, Venkatakrishnan V (2019) Holmes: real-time apt detection through correlation of suspicious information flows. In: 2019 IEEE symposium on security and privacy (SP), IEEE, pp 1137–1152. <https://doi.org/10.1109/SP2019.00026>
- Mo Y, Nie F, Hu P, Shen H.T, Zhang Z, Wang X, Zhu X (2024) Self-supervised heterogeneous graph learning: a homophily and heterogeneity view. In: The twelfth international conference on learning representations. <https://openreview.net/forum?id=3FJOKjoolj>
- Narayanan A, Chandramohan M, Venkatesan R, Chen L, Liu Y, Jaiswal S (2017) graph2vec: learning distributed representations of graphs. *arXiv:1707.05005*<https://doi.org/10.48550/arXiv.1707.05005>
- Pasquier T, Han X, Goldstein M, Moyer T, Eysers D, Seltzer M, Bacon J (2017) Practical whole-system provenance capture. In: Proceedings of the 2017 symposium on cloud computing, pp 405–418
- Paszke A, Gross S, Massa F, Lerer A, Bradbury J, Chanan G, Killeen T, Lin Z, Gimelshein N, Antiga L, et al (2019) Pytorch: an imperative style, high-performance deep learning library. *Advances in neural information processing systems*. 32
- Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, Blondel M, Prettenhofer P, Weiss R, Dubourg V et al (2011) Scikit-learn: machine learning in python. *J Mach Learn Res* 12:2825–2830
- Petya (2024) In Wikipedia, the free encyclopedia. Available from [https://en.wikipedia.org/wiki/Petya_\(malware_family\)](https://en.wikipedia.org/wiki/Petya_(malware_family)). Accessed 18 Sept 2024
- Rehman MU, Ahmadi H, Hassan WU (2024) Flash: a comprehensive approach to intrusion detection via provenance graph representation learning. In: 2024 IEEE symposium on security and privacy (SP), IEEE Computer Society, pp 139–139. <https://doi.org/10.1109/SP54263.2024.00139>
- Shen Y, Stringhini G (2019) ATTACK2VEC: leveraging temporal word embeddings to understand the evolution of cyberattacks. In: 28th USENIX security symposium (USENIX Security 19), pp 905–921. USENIX Association, Santa Clara, CA. <https://www.usenix.org/conference/usenixsecurity19/presentation/shen>
- Si S, Zhang H, Keerthi SS, Mahajan D, Dhillon IS, Hsieh C-J (2017) Gradient boosted decision trees for high dimensional sparse output. In: International conference on machine learning, PMLR, pp 3182–3190
- Vaswani A (2017) Attention is all you need. *Advances in Neural Information Processing Systems* <https://doi.org/10.48550/arXiv.1706.03762>
- Wang S, Wang Z, Zhou T, Sun H, Yin X, Han D, Zhang H, Shi X, Yang J (2022) Threatrace: detecting and tracing host-based threats in node level through provenance graph learning. *IEEE Trans Inf Forensics Secur* 17:3972–3987. <https://doi.org/10.1109/TIFS.2022.3208815>
- Wang Q, Hassan WU, Li D, Jee K, Yu X, Zou K, Rhee J, Chen Z, Cheng W, Gunter CA, et al (2020) You are what you do: Hunting stealthy malware via data provenance analysis. In: NDSS. <https://doi.org/10.14722/ndss.2020.24167>
- Wang M, Zheng D, Ye Z, Gan Q, Li M, Song X, Zhou J, Ma C, Yu L, Gai Y, et al (2019) Deep graph library: a graph-centric, highly-performant package for graph neural networks. *arXiv:1909.01315*
- Woodbury MA (1950) Inverting modified matrices. Department of Statistics, Princeton University
- Xia B, Yin J, Xu J, Li Y (2019) Loggan: a sequence-based generative adversarial network for anomaly detection based on system logs. In: Science of cyber security: second international conference, SciSec 2019, Springer, Nanjing, China, August 9–11, 2019, Revised Selected Papers 2, pp. 61–76. https://doi.org/10.1007/978-3-030-34637-9_5
- Xiao N, Lang B, Wang T, Chen Y (2024) Apt-mm: an advanced persistent threat actor attribution method based on multimodal and multilevel feature fusion. *Comput Secur* 144:103960. <https://doi.org/10.1016/j.cose.2024.103960>
- Xu Z, Fang P, Liu C, Xiao X, Wen Y, Meng D (2022) Depcomm: graph summarization on system audit logs for attack investigation. In: 2022 IEEE symposium on security and privacy (SP), IEEE, pp 540–557. <https://doi.org/10.1109/SP46214.2022.9833632>
- Yang F, Xu J, Xiong C, Li Z, Zhang K (2023) PROGRAPHER: an anomaly detection system based on provenance graph embedding. In: 32nd USENIX security symposium (USENIX Security 23), pp 4355–4372. USENIX Association, Anaheim, CA. <https://www.usenix.org/conference/usenixsecurity23/presentation/yang-fan>
- Zengy J, Wang X, Liu J, Chen Y, Liang Z, Chua T-S, Chua ZL (2022) Shade-watcher: recommendation-guided cyber threat analysis using system audit records. In: 2022 IEEE symposium on security and privacy (SP), IEEE, pp 489–506. <https://doi.org/10.1109/SP46214.2022.9833669>
- Zhang X, Xu Y, Lin Q, Qiao B, Zhang H, Dang Y, Xie C, Yang X, Cheng Q, Li Z, et al (2019) Robust log-based anomaly detection on unstable log data. In: Proceedings of the 2019 27th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering, pp 807–817. <https://doi.org/10.1145/3338906.3338931>

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.